# 2 Algorithmique

### Prérequis à réactiver

Effectuer des calculs dans le bon ordre; lire un tableau à double entrée; suivre une consigne composée de plusieurs étapes.

### Automatismes

Partir de $x = 3$ et exécuter successivement : multiplier par 2, ajouter 5, remplacer le résultat par son carré. Recommencer avec $x = -1$.

### Activité d'entrée – Faire exécuter sans ambiguïté

Un élève décrit oralement la construction d'une figure très simple sans la montrer; un autre l'exécute. Comparer la figure obtenue à la figure cible et repérer les instructions ambiguës. Réécrire la procédure avec des données d'entrée, des instructions ordonnées et un résultat contrôlable.

### *L'origine du mot « algorithme »*

*Le mot **algorithme** vient du nom du mathématicien perse Muhammad ibn Mûsâ **al-Khwârizmî** (780–850), dont le nom latinement translittéré donna Algoritmi. Ce même savant donna son nom à une autre discipline : l'**algèbre**, de al-jabr, titre de son traité fondateur. Un homme, deux sciences.*

### Objectifs du chapitre

- Comprendre la notion d'algorithme et ses propriétés.
- Lire, exécuter et tracer un algorithme en pseudo-code.

- Tracer et lire un organigramme.
- Écrire un algorithme simple pour un problème mathématique.

## 2.1 Qu'est-ce qu'un algorithme ?

**Définition 2.1 – *Algorithme***

Un **algorithme** est une suite finie et non ambiguë d'instructions permettant de résoudre un problème. Il doit être : **finie** (il termine), **valide** (il donne le bon résultat), et **déterministe** (même entrée $\Rightarrow$ même sortie).

## 2.2 Variables, affectation, séquence

**Définition 2.2 – *Variable et affectation***

Une **variable** est un emplacement mémoire identifié par un nom. L'instruction $A \leftarrow 5$ signifie « affecter la valeur 5 à $A$ ». Attention : $A \leftarrow A+1$ remplace l'ancienne valeur de $A$ par (ancienne valeur) + 1.

**Exemple 2.1 – *Trace d'algorithme***

Exécuter : $A \leftarrow 3$, $B \leftarrow A+7$, $A \leftarrow A \times 2$

| Étape | Instruction | $A$ | $B$ |
|---|---|---|---|
| 1 | $A \leftarrow 3$ | 3 | – |
| 2 | $B \leftarrow A+7$ | 3 | 10 |
| 3 | $A \leftarrow A \times 2$ | 6 | 10 |

## 2.3 Structures de contrôle

**Définition 2.3 – *Structures de base***

- **Séquence** : les instructions s'exécutent dans l'ordre.
- **Alternative** (`Si ...Sinon`) : exécution conditionnelle.

- **Itération** (`Tant que` / `Pour`) : répétition.

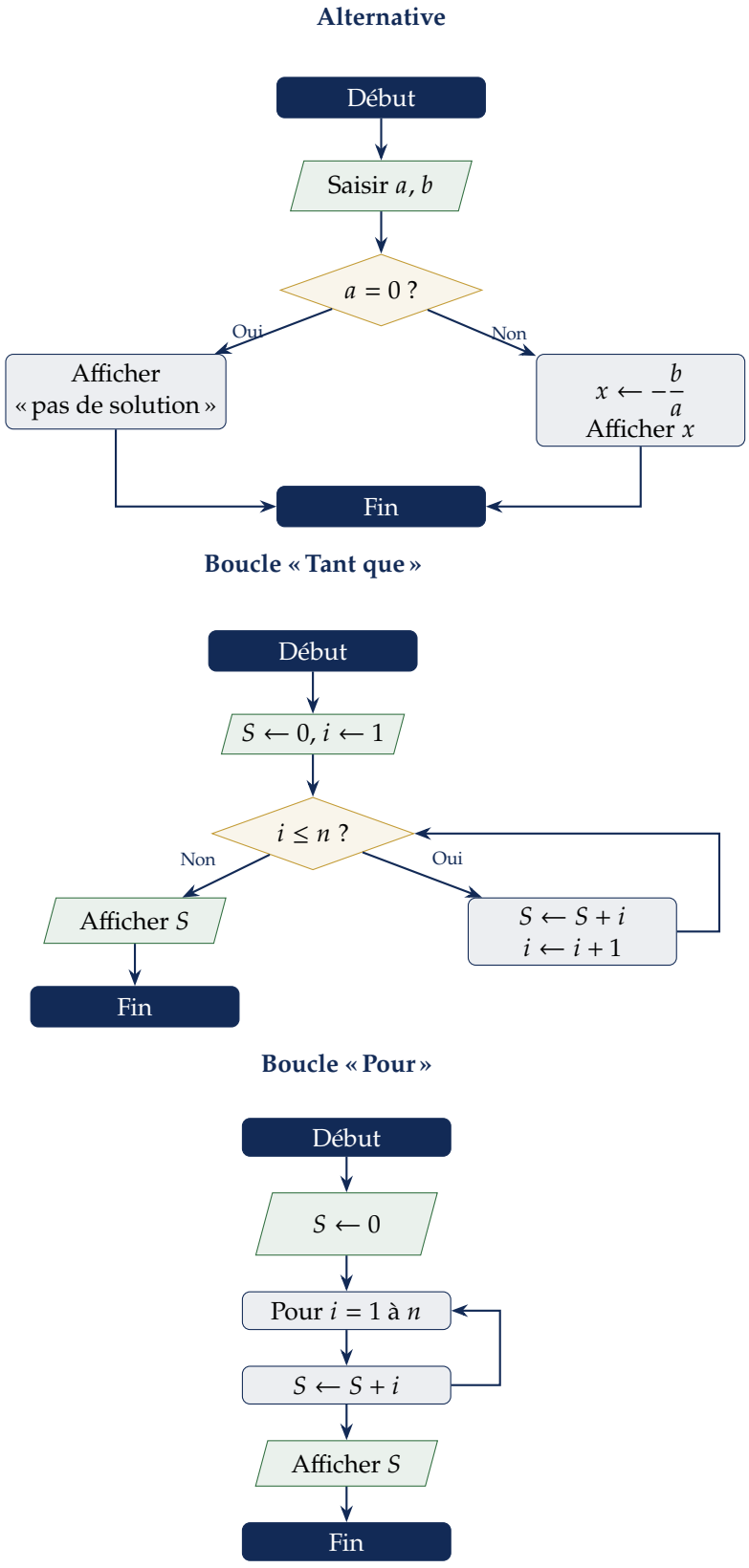



Figure 2.1: Les trois structures de contrôle en organigramme. **Alternative** : exécute l'une ou l'autre branche. **Tant que** : répète tant que la condition est vraie (test au début). **Pour** : répète un nombre fixé de fois.

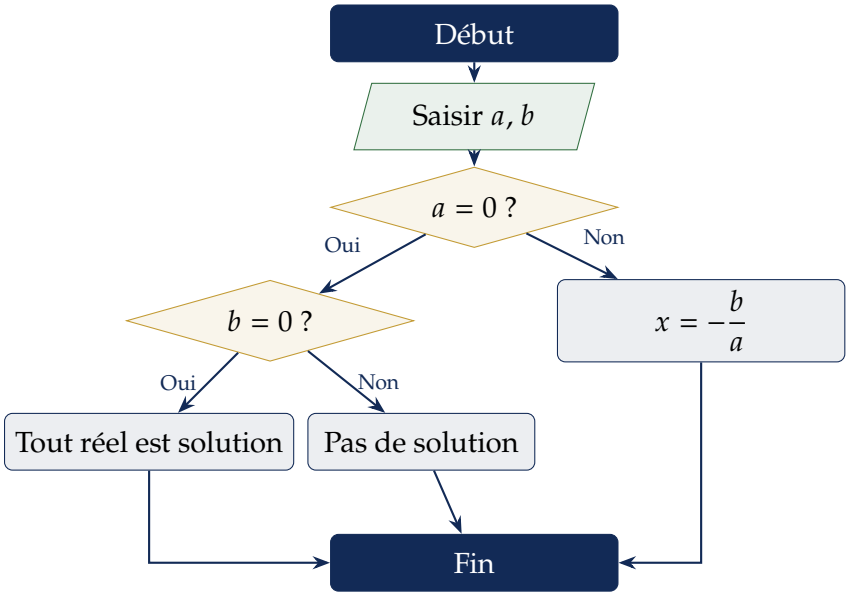



Figure 2.2: Organigramme : résolution de $ax + b = 0$.

Le même algorithme en pseudo-code :

**Algorithme 2 – Résolution de $ax + b = 0$**

**Entrée :** deux réels $a$ et $b$
**Sortie :** la (ou les) solution(s) de $ax + b = 0$
**Lire** $a$, $b$;
**Si** $a = 0$ **alors**
  **Si** $b = 0$ **alors**
    **Afficher** « Tout réel est solution »;
  **Sinon**
    **Afficher** « Pas de solution »;
  **Fin si**
**Sinon**
  $x \leftarrow -b/a$;
  **Afficher** « Solution unique : $x =$ », $x$;
**Fin si**

**Hors programme T10 – non exigible – Début d'un approfondissement**

**Tout le développement signalé ci-après est hors programme T10 : ne pas l'enseigner comme prérequis et ne pas l'évaluer.** Cette partie propose des algorithmes plus longs qui combinent variables, conditions et boucles.

## 2.4 Méthodes et compléments de cours

### Méthode 2.1 – *Construire la trace d'exécution*

Pour *exécuter à la main* un algorithme :

1. Créer un tableau avec une colonne par variable.
2. Exécuter chaque instruction dans l'ordre, ligne par ligne.
3. À chaque affectation, mettre à jour la colonne concernée en barrant l'ancienne valeur.
4. Pour un `Si/Sinon`, évaluer la condition *avant* de choisir la branche.
5. Pour un `Tant que`, répéter l'évaluation de la condition à chaque itération.

### Méthode 2.2 – *Conventions de pseudo-code*

Dans ce manuel, on utilise les conventions suivantes :

- **Affectation :** $X \leftarrow$ expression
- **Lecture :** `Lire`$(X)$ (saisie utilisateur)
- **Écriture :** `Afficher`$(X)$
- **Alternative :** `Si` *condition* `alors ... Sinon ... Fin Si`
- **Boucle bornée :** `Pour` $i$ `de` $a$ `à` $b$ `faire ... Fin Pour`
- **Boucle non bornée :** `Tant que` *condition* `faire ... Fin Tant que`

### Définition 2.4 – *Invariant de boucle*

Un **invariant de boucle** est une propriété vraie avant la boucle, conservée à chaque itération, et qui permet de prouver la correction de l'algorithme à la sortie.

*Remarque.* Le **coût** d'un algorithme se mesure en comptant le nombre d'opérations élémentaires effectuées en fonction de la taille $n$ de l'entrée. Plus ce nombre croît lentement avec $n$, plus l'algorithme est efficace. On parle d'algorithme *rapide* (peu d'opérations) ou *lent* (beaucoup d'opérations).

**ADA LOVELACE, PREMIÈRE PROGRAMMEUSE**

*En 1843, Ada Lovelace (1815–1852) publia ce qui est considéré comme le premier algorithme destiné à être exécuté par une machine : un programme pour calculer les nombres de Bernoulli sur la* machine analytique *de Babbage, qui ne fut jamais construite. Elle imagina aussi que la machine pourrait composer de la musique, un siècle avant l'ordinateur.*

## 2.5 Exercices

### ★ Niveau 1 — Traces et écriture directe

**Exercice 2.1** ALGO *Trace simple* ★

Donner la trace d'exécution et les valeurs finales de $A$, $B$, $C$ :

$$A \leftarrow 5 \quad B \leftarrow 3 \quad C \leftarrow A + B \quad A \leftarrow A \times B \quad B \leftarrow C - A$$

**Exercice 2.2** ALGO *Trace avec variables supplémentaires* ★

Donner la trace et les valeurs finales :

$$X \leftarrow 10 \quad Y \leftarrow 3 \quad Z \leftarrow X \div Y \quad R \leftarrow X - Z \times Y \quad X \leftarrow Z$$

*(L'opérateur $\div$ désigne ici la division entière.)*

**Exercice 2.3** ALGO *Échange de deux variables* ★

On veut échanger les valeurs de $A$ et $B$.

1. Montrer que le code suivant est incorrect : $A \leftarrow B$; $B \leftarrow A$.
2. Écrire un algorithme correct utilisant une variable temporaire $T$.
3. Donner la trace pour $A = 7$, $B = 3$.

**Exercice 2.4** *Calcul de moyenne* ★

Écrire en pseudo-code puis implémenter Python un algorithme qui :

1. Saisit trois notes $n_1, n_2, n_3$ (entre 0 et 20).
2. Calcule et affiche leur moyenne.
3. Affiche « Reçu » si la moyenne est $\geq 10$, « Recalé » sinon.

**Exercice 2.5** *Équation du premier degré* ★

Écrire un algorithme qui saisit $a$ et $b$, puis résout $ax + b = 0$ en gérant les trois cas : $a \neq 0$ (une solution), $a = 0$ et $b = 0$ (infinité de solutions), $a = 0$ et $b \neq 0$ (pas de solution).

**Exercice 2.6** *Organigramme pair/impair* ★

1. Dessiner l'organigramme d'un algorithme qui saisit un entier $n$ et affiche « pair » ou « impair ».
2. Écrire le pseudo-code correspondant.

**Exercice 2.7** *Valeur absolue* ★

Écrire un algorithme qui saisit un réel $x$ et affiche $|x|$ sans utiliser de fonction prédéfinie. Tester pour $x = -5$, $x = 0$, $x = 3{,}14$.

**Exercice 2.8** *Maximum de trois nombres* ★

Écrire un algorithme qui saisit trois réels $a, b, c$ et affiche le plus grand des trois.

1. Version avec des `Si` imbriqués.
2. Version en utilisant une variable `Max`.

**Exercice 2.9** *Table de multiplication* ★

Écrire un algorithme qui saisit un entier $n \geq 1$ et affiche sa table de multiplication de $1 \times n$ à $10 \times n$. Donner la trace pour $n = 3$, puis implémenter en Python Python.

**Exercice 2.10** *Lecture d'algorithme* ★

Que calcule et affiche cet algorithme ?

> **Lire** $N$ ;
> $S \leftarrow 0$ ;
> **Pour** $i \leftarrow 1$ **à** $N$ **faire**
>  $S \leftarrow S + i$ ;
> **Fin pour**
> **Afficher** $S$ ;

Vérifier pour $N = 5$.

**Exercice 2.11** *Compter les multiples* ★

Écrire un algorithme qui affiche tous les multiples de 3 strictement inférieurs à 100. Combien y en a-t-il ?

**Exercice 2.12** *Conversion Celsius/Fahrenheit* ★

La conversion est $F = \frac{9}{5}C + 32$. Écrire un algorithme qui affiche un tableau de conversion pour $C \in \{0, 10, 20, \ldots, 100\}$.

**Exercice 2.13** *Puissance entière* ★

Écrire un algorithme qui saisit $x$ et $n \geq 0$ et calcule $x^n$ par multiplications successives (sans utiliser d'opérateur puissance). Tester pour $x = 2$, $n = 8$.

**Exercice 2.14** *Somme des chiffres d'un entier* ★

Écrire un algorithme qui saisit un entier $n > 0$ et affiche la somme de ses chiffres. *Indication : utiliser la division euclidienne par* 10. Tester pour $n = 1234$.

**Exercice 2.15** *Lecture de trace* ★

Exécuter cet algorithme pour $A = 4$, $B = 6$ et décrire ce qu'il calcule :

```
Entrée : deux entiers A, B > 0
Sortie : un entier
LireA, B;
Tant que B ≠ 0 faire
    R ← A mod B                    // reste de la division de A par B
    A ← B;
    B ← R;
Fin tant que
AfficherA;
```

*(Rappel : $A \mod B$ est le reste de la division de $A$ par $B$.)*

## ★★ Niveau 2 — Boucles et structures combinées

**Exercice 2.16** *Suite arithmétique* ★★

Écrire un algorithme qui saisit $u_0$ et $r$, puis affiche les 20 premiers termes de la suite arithmétique $u_n = u_0 + nr$. Calculer aussi la somme $S = u_0 + u_1 + \cdots + u_{19}$ et vérifier avec la formule $S = 20u_0 + 190r$.

**Exercice 2.17** *Suite géométrique et seuil* ★★

On considère la suite géométrique $u_n = 3 \times 2^n$. Écrire un algorithme puis l'implémenter Python pour :

1. Afficher les 10 premiers termes.
2. Trouver le plus petit $n$ tel que $u_n > 1000$.

Vérifier algébriquement le résultat de (2).

**Exercice 2.18** ALGO *Factorielle* ★★

Écrire un algorithme calculant $n!$ pour un entier $n \geq 0$ saisi.

1. Version itérative (`Pour`).
2. Ajouter une vérification : si $n < 0$, afficher un message d'erreur.
3. Combien de multiplications effectue l'algorithme ?

**Exercice 2.19** *Maximum et position* ★★

Écrire un algorithme qui lit une séquence de $n$ réels (un par un) et retourne :

- La valeur maximale.
- Sa position dans la séquence (rang du premier maximum trouvé).

Combien de comparaisons effectue-t-il ?

**Exercice 2.20** *Recherche linéaire* ★★

Écrire un algorithme qui lit $n$ entiers puis une valeur cible $v$, et affiche la position de la première occurrence de $v$ (ou un message si $v$ est absent).

1. Version avec `Tant que`.
2. Combien de comparaisons dans le pire cas ?

**Exercice 2.21** *Palindrome* ★★

Un mot est un *palindrome* s'il se lit de la même façon dans les deux sens (« *kayak* », « *radar* », etc.).
Écrire un algorithme qui saisit un mot (une chaîne de caractères) et détermine si c'est un palindrome. *Indication : comparer le $i$-ème et le $(n-i)$-ème caractère.*

**Exercice 2.22** *Suite de Syracuse* ★★

La suite de Syracuse part d'un entier $n > 0$ :

$$u_{k+1} = \begin{cases} \dfrac{u_k}{2} & \text{si } u_k \text{ est pair,} \\ 3u_k + 1 & \text{si } u_k \text{ est impair.} \end{cases}$$

Écrire un algorithme qui :

1. Affiche tous les termes jusqu'à ce que $u_k = 1$.
2. Compte le nombre de termes (le *temps de vol*).
3. Calcule le maximum atteint (le *vol*).

Tester pour $n = 6$ et $n = 27$.

**Exercice 2.23** *Diviseurs d'un entier* ★★

Écrire un algorithme qui saisit $n > 0$ et affiche tous ses diviseurs.

1. Version testant tous les entiers de 1 à $n$.
2. Version optimisée testant de 1 à $\lfloor \sqrt{n} \rfloor$ (et affichant les diviseurs par paires).
3. Comparer le nombre de tests dans les deux versions pour $n = 360$.

**Exercice 2.24** ALGO *Nombre premier* ★★

Écrire un algorithme qui teste si un entier $n \geq 2$ est premier.

1. Justifier pourquoi il suffit de tester les diviseurs jusqu'à $\sqrt{n}$.
2. Écrire le pseudo-code.
3. Afficher tous les nombres premiers inférieurs à 100, puis implémenter en Python Python.

**Exercice 2.25** *Zéro d'une fonction par dichotomie* Python ★★

On cherche à approcher une racine de $f(x) = x^3 - 2$ sur $[1, 2]$.

1. Vérifier que $f(1) < 0$ et $f(2) > 0$ (théorème des valeurs intermédiaires).
2. Écrire un algorithme de *dichotomie* : tant que $b - a > \varepsilon$, calculer $m = \dfrac{a+b}{2}$ et mettre à jour $[a, b]$.
3. Combien d'itérations pour $\varepsilon = 10^{-6}$ ?

4. Comparer avec $\sqrt[3]{2}$.

---

**Exercice 2.26** *Codage de César* ★★

Le chiffrement de César décale chaque lettre de $k$ rangs dans l'alphabet.

1. Écrire un algorithme qui chiffre un message lettre par lettre (on supposera les lettres codées de 0 à 25).
2. Écrire l'algorithme de déchiffrement.
3. Quel est le lien avec les calculs modulo 26 ?
4. Pourquoi ce chiffrement est-il considéré comme non sécurisé ?

---

**Exercice 2.27** *Alignement de points* ★★

Écrire un algorithme qui saisit trois points $A(x_A, y_A)$, $B(x_B, y_B)$, $C(x_C, y_C)$ et détermine s'ils sont alignés. *Rappel : $A$, $B$, $C$ sont alignés ssi $\overrightarrow{AB} \times \overrightarrow{AC} = 0$, i.e. $(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A) = 0$.* Tester avec $A(0,0)$, $B(1,2)$, $C(2,4)$ et $A(0,0)$, $B(1,2)$, $C(2,5)$.

---

**Exercice 2.28** *Calcul de $\pi$ par série* ★★

La série de Leibniz donne : $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \cdots = \displaystyle\sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$.

1. Écrire un algorithme qui calcule la somme des $N$ premiers termes.
2. Afficher l'approximation de $\pi$ pour $N = 10, 100, 1000, 10000$.
3. Cette série converge-t-elle vite ? Combien de termes pour avoir 5 décimales correctes ?

---

**Exercice 2.29** *Schéma de Hörner* ★★

Évaluer le polynôme $P(x) = a_n x^n + \cdots + a_1 x + a_0$ en un point $x_0$ par le schéma de Hörner :

$$P(x_0) = (\ldots((a_n x_0 + a_{n-1})x_0 + a_{n-2})\ldots)x_0 + a_0.$$

1. Écrire l'algorithme (les coefficients sont lus de $a_n$ à $a_0$).
2. Combien de multiplications effectue-t-il ? Comparer avec la méthode naïve.
3. Appliquer à $P(x) = 2x^3 - 5x^2 + 3x - 1$ en $x_0 = 2$.

**Hors programme T10 – non exigible – Début d'un approfondissement**

**Tout le développement signalé ci-après est hors programme T10 : ne pas l'enseigner comme prérequis et ne pas l'évaluer.** Les suites, la preuve par invariant, l'analyse du nombre d'opérations et les algorithmes spécialisés qui suivent sont facultatifs.

## ★★★ Niveau 3 — Raisonnement et algorithmes avancés

### Exercice 2.30 ALGO *Suite de Fibonacci et rapport doré* Python ★★★

Écrire un algorithme calculant les $N$ premiers termes de la suite de Fibonacci ($u_0 = 0$, $u_1 = 1$, $u_{n+1} = u_n + u_{n-1}$).

1. Afficher aussi le rapport $\dfrac{u_{n+1}}{u_n}$ pour $n \geq 1$.

2. Vers quelle valeur ce rapport semble-t-il converger ? (C'est le nombre d'or $\varphi = \dfrac{1+\sqrt{5}}{2}$.)

3. Modifier l'algorithme pour s'arrêter dès que ce rapport diffère de $\varphi$ de moins de $10^{-6}$.

### Exercice 2.31 ALGO *Preuve de correction : invariant* ★★★

Considérons l'algorithme de calcul de $n!$ :

**Algorithme 5 – Calcul itératif de $n!$**

**Entrée :** un entier $n \geq 0$
**Sortie :** $n!$
**Lire** $n$;
$F \leftarrow 1$;
$k \leftarrow 1$;
**Tant que** $k \leq n$ **faire**
  $F \leftarrow F \times k$;
  $k \leftarrow k + 1$;
**Fin tant que**
**Afficher** $F$;

1. Vérifier que $F = (k-1)!$ est un invariant de boucle.

2. En déduire la valeur de $F$ à la sortie de la boucle.

3. L'algorithme est-il correct ? Termine-t-il toujours ?

**Exercice 2.32** *Algorithme d'Euclide* ★★★

L'algorithme d'Euclide calcule $\text{pgcd}(a, b)$ : tant que $b \neq 0$, poser $r \leftarrow a \mod b$, $a \leftarrow b$, $b \leftarrow r$.

1. Écrire le pseudo-code et donner la trace pour $a = 48$, $b = 18$.
2. Démontrer la correction : montrer que $\text{pgcd}(a, b) = \text{pgcd}(b, a \mod b)$.
3. Montrer que l'algorithme termine (le reste strictement décroît).
4. Combien d'étapes pour $a = F_{n+1}$, $b = F_n$ (nombres de Fibonacci) ?

**Exercice 2.33** ALGO *Exponentiation rapide* ★★★

La méthode naïve calcule $x^n$ en $n - 1$ multiplications. L'*exponentiation rapide* (squaring) utilise :

$$x^n = \begin{cases} 1 & n = 0, \\ (x^{n/2})^2 & n \text{ pair}, \\ x \cdot x^{n-1} & n \text{ impair}. \end{cases}$$

1. Écrire l'algorithme itératif correspondant.
2. Donner la trace pour $x = 2$, $n = 13$.
3. Combien de multiplications effectue-t-il en fonction de $n$ ?
4. Comparer avec la méthode naïve pour $n = 1000$.

**Exercice 2.34** ALGO *Tri bulles* ★★★

Le *tri à bulles* parcourt le tableau en comparant chaque paire d'éléments adjacents et en les échangeant si nécessaire, répétant jusqu'à ce que le tableau soit trié.

1. Écrire le pseudo-code du tri à bulles pour un tableau de $n$ éléments.
2. Donner la trace pour $[5, 3, 8, 1, 9, 2]$.
3. Combien de comparaisons dans le pire cas ?
4. Proposer une optimisation : s'arrêter si aucun échange n'a eu lieu.

**Exercice 2.35** ALGO *Recherche dichotomique* ★★★

Dans un tableau trié de $n$ éléments, la *recherche dichotomique* compare la valeur cherchée avec l'élément du milieu et réduit le domaine de recherche de moitié à chaque étape.

1. Écrire le pseudo-code.

2. Donner la trace pour la recherche de 17 dans $[2, 5, 8, 12, 17, 23, 38]$.

3. Pour $n = 8$ : combien d'itérations sont nécessaires ? Et pour $n = 16$, $n = 64$, $n = 1024$ ? Remplir le tableau :

| $n$ | Dichotomie | Linéaire (pire cas) |
|---|---|---|
| 8 | | |
| 16 | | |
| 64 | | |
| 1024 | | |

4. Pour $n = 10^6$ : estimer le nombre d'itérations de chaque méthode.

**Exercice 2.36** *Codage et décodage binaire* ★★★

1. Écrire un algorithme qui convertit un entier $n \geq 0$ en binaire (afficher les bits de poids faible au poids fort en utilisant la division euclidienne par 2).

2. Écrire l'algorithme inverse (binaire vers décimal).

3. Tester pour $n = 42$.

4. Combien de bits faut-il pour représenter un entier $n$ ?

## Problèmes

**Problème 2.1 – *Simulation d'un jeu de dés*** ★

On simule le lancer de deux dés à 6 faces. Le score est la somme des deux résultats.

1. Écrire l'algorithme ci-dessous en pseudo-code, puis l'implémenter Python :

**Algorithme 6 – Simulation de $N$ lancers de dés**

**Entrée :** un entier $N$ (nombre de lancers)
**Sortie :** le nombre de fois où la somme vaut 7

```
compte ← 0;
Pour i ← 1 à N faire
    d1 ← Hasard(1, 6)                    // entier aléatoire entre 1 et 6
    d2 ← Hasard(1, 6);
    Si d1 + d2 = 7 alors
        compte ← compte + 1;
    Fin si
Fin pour
Afficher compte;
```

2. Calculer théoriquement la probabilité d'obtenir 7. Vérifier avec votre simulation pour $N = 1000$.
3. Modifier l'algorithme pour afficher la distribution complète des sommes possibles (de 2 à 12).

### Problème 2.2 – *Suite de Collatz et conjecture* ★★

La suite de Collatz est $u_{n+1} = \frac{u_n}{2}$ si $u_n$ pair, $3u_n + 1$ sinon. La conjecture (non prouvée !) affirme qu'elle atteint toujours 1.

1. Écrire un algorithme qui calcule le temps de vol (nombre de pas avant d'atteindre 1) pour tout $n$ donné.
2. Trouver par ordinateur la valeur $n \leq 1000$ qui maximise le temps de vol. Quelle est cette valeur maximale ?
3. Pourquoi ne peut-on pas « prouver » la conjecture par cet algorithme, même en testant des millions de cas ?

### Problème 2.3 – *Méthode de Héron : racine carrée* ★

La méthode de Héron approche $\sqrt{a}$ par la suite : $u_0 = a$, $\quad u_{n+1} = \frac{1}{2}\left(u_n + \frac{a}{u_n}\right)$.

1. Écrire l'algorithme suivant et le tester Python :

**Algorithme 7 – Approximation de $\sqrt{a}$ par la méthode de Héron**

**Entrée :** un réel $a > 0$, une précision $\varepsilon > 0$
**Sortie :** une approximation de $\sqrt{a}$
$u \leftarrow a$;
**Répéter**
  $u_{\text{new}} \leftarrow \frac{1}{2}\left(u + \frac{a}{u}\right)$;
  $u \leftarrow u_{\text{new}}$;
**Jusqu'à** $|u_{\text{new}} - u| < \varepsilon$
**Afficher**$u$;

2. Calculer à la main les 4 premiers termes pour $a = 2$. Comparer avec $\sqrt{2} \approx 1{,}41421$.
3. La suite converge-t-elle vite ? Compte le nombre d'itérations pour $\varepsilon = 10^{-6}$.
4. Démontrer algébriquement que si $u_n > 0$, alors $u_{n+1} \geq \sqrt{a}$. *(Indication : inégalité arithmético-géométrique $\frac{x+y}{2} \geq \sqrt{xy}$ pour $x, y > 0$.)*

## Problème 2.4 – *Algorithme de Horner et évaluation de polynômes* ★★

On considère $P(x) = 3x^4 - 7x^3 + 2x^2 + x - 5$.

1. Calculer $P(2)$ en développant directement. Compter le nombre de multiplications effectuées.
2. Le schéma de Hörner réécrit : $P(x) = ((((3)x - 7)x + 2)x + 1)x - 5$. Calculer $P(2)$ en appliquant les parenthèses de l'intérieur vers l'extérieur. Compter les multiplications.

**Algorithme 8 – Schéma de Hörner pour $P(x_0) = a_n x^n + \cdots + a_0$**

**Entrée :** les coefficients $a_n, a_{n-1}, \ldots, a_0$ et un réel $x_0$
**Sortie :** $P(x_0)$
$val \leftarrow a_n$;
**Pour** $k \leftarrow n - 1$ **à**$0$ **faire**
  $val \leftarrow val \times x_0 + a_k$;
**Fin pour**
**Afficher**$val$;

3. Appliquer l'algorithme à la main pour $P(x) = 3x^4 - 7x^3 + 2x^2 + x - 5$, $x_0 = 2$. Vérifier que vous obtenez le même résultat qu'en (1).
4. Implémenter cet algorithme Python et tester sur $P(x) = x^3 - 6x^2 + 11x - 6$ pour $x_0 \in \{1, 2, 3\}$.

### Problème 2.5 – *Cryptographie par décalage : attaque* ★★

Un message est chiffré par décalage de César avec une clé $k$ inconnue.

1. Écrire un algorithme qui essaie toutes les clés de 0 à 25 et affiche les 26 déchiffrements possibles.
2. Comment identifier automatiquement le bon déchiffrement ? *(Indication : compter la lettre « e », la plus fréquente en français.)*
3. Cet algorithme d'attaque est dit *force brute*. Pourquoi est-il inapplicable contre des chiffrements modernes ?

### Problème 2.6 – *Jeu du plus/moins* ★

L'ordinateur choisit un entier aléatoire $n$ entre 1 et 100. L'utilisateur propose des valeurs ; l'ordinateur répond « plus », « moins » ou « trouvé ».

1. Écrire l'algorithme du jeu.
2. Quelle stratégie minimise le nombre de tentatives ?
3. Montrer que la stratégie optimale (dichotomie) réussit en au plus $\lceil \log_2 100 \rceil = 7$ tentatives.
4. Modifier l'algorithme pour compter les tentatives.

### Problème 2.7 – *Crible d'Ératosthène* ★★

Le *crible d'Ératosthène* liste tous les nombres premiers jusqu'à $N$ : on initialise un tableau de booléens à **V**, puis pour chaque premier $p$, on marque à **F** ses multiples $p^2, p^2 + p, p^2 + 2p, \ldots$

1. Écrire l'algorithme en pseudo-code.
2. Donner la trace pour $N = 30$.
3. Pour $N = 100$ : compter le nombre total de cases marquées pendant l'exécution. L'algorithme est-il rapide ?
4. Pourquoi commence-t-on le marquage à $p^2$ plutôt qu'à $2p$ ?

### Problème 2.8 – *Tri fusion : principe* ★★★

Le *tri fusion* divise le tableau en deux moitiés, les trie récursivement, puis les fusionne.

1. Écrire l'algorithme de *fusion* de deux tableaux triés.
2. Écrire le schéma récursif du tri fusion (on peut décrire la récursion sans la coder entièrement).
3. Appliquer sur $[5, 3, 8, 1, 9, 2, 4, 7]$ en détaillant les étapes.
4. Pour $n = 8$ éléments, compter le nombre total de comparaisons. Le tri fusion est-il plus rapide que le tri à bulles sur cet exemple ?

### Problème 2.9 – *Chiffrement de Vigenère* ★★

Le chiffrement de Vigenère est une extension du chiffre de César : on utilise une clé de plusieurs lettres. Si la clé est « CLÉ » (rang 2, 11, 4 dans l'alphabet), on décale le $i$-ème caractère du message par le $i$-ème rang de la clé (cycliquement).

1. Chiffrer le message « BONJOUR » avec la clé « MATHS ». *(A=0, B=1, . . . , Z=25 ; appliquer le décalage modulo 26.)*
2. Écrire l'algorithme de chiffrement Python.
3. Écrire l'algorithme de déchiffrement.
4. Pourquoi ce chiffrement est-il plus sûr que le chiffre de César seul ? Combien de clés de longueur 5 existe-t-il ?

### Problème 2.10 – *Algorithme de compression RLE* ★★

Le *Run-Length Encoding* (RLE) compresse une séquence en remplaçant les suites de caractères identiques par le caractère et son nombre de répétitions (ex. : `AAABBBBA` → `3A4B1A`).

1. Écrire l'algorithme de compression.
2. Écrire l'algorithme de décompression.
3. Tester sur `AAAAAABBBCCCCCD`.
4. Dans quels cas ce codage est-il inefficace ?

### Problème 2.11 – L&A *Correction logique d'un algorithme* ★★

Voici un algorithme censé calculer le maximum d'une liste de $n$ valeurs :

**Algorithme 9 – Maximum (avec bug)**

$max \leftarrow 0$;
**Pour** $i \leftarrow 1$ **à** $n$ **faire**
  **Si** $L[i] > max$ **alors**
    $max \leftarrow L[i]$;
  **Fin si**
**Fin pour**
**Afficher** $max$;

1. Tester avec $L = \{3, 7, 2, 9, 5\}$ : donne-t-il le bon résultat ?
2. Tester avec $L = \{-4, -1, -7, -2\}$ : est-ce correct ? Si non, identifier logiquement l'erreur.
3. La proposition « l'algorithme calcule correctement le maximum pour toute liste » est-elle vraie ou fausse ? Donner un contre-exemple si fausse.
4. Corriger l'algorithme pour qu'il soit correct pour n'importe quelle liste.
5. Énoncer sous forme logique l'invariant correct : quelle propriété `max` doit-il vérifier après chaque itération ?

### Problème 2.12 – *Calcul de $\pi$ par méthode de Monte-Carlo* ★★

On tire aléatoirement des points $(x, y)$ dans $[0, 1]^2$. Un point est dans le quart de disque unité si $x^2 + y^2 \leq 1$. La proportion de tels points approche $\pi/4$.

1. Écrire l'algorithme simulant $N$ tirages et estimant $\pi$.
2. Tester pour $N = 100, 1000, 10^6$.
3. Estimer l'erreur en fonction de $N$.
4. Pourquoi appelle-t-on cela la *méthode de Monte-Carlo* ?

### Problème 2.13 – *Statistiques par algorithme* Python ★★

Écrire un programme Python qui :

1. Lit une liste de $n$ notes (entre 0 et 20) saisies par l'utilisateur.
2. Calcule et affiche la moyenne, le minimum, le maximum et l'étendue.
3. Calcule et affiche la médiane (après tri de la liste).

4. Trace un histogramme des notes regroupées par tranches : $[0, 5[$, $[5, 10[$, $[10, 15[$, $[15, 20]$.
5. Bonus : afficher si la classe a une moyenne $\geq 10$ ou $< 10$.

*On pourra utiliser les fonctions* `sorted()`, `len()`, *et la bibliothèque* `matplotlib` *pour le tracé.*

### Défis

**★ Défi – hors programme T10, non exigible – *Tri par sélection***

Le *tri par sélection* cherche le minimum dans la partie non triée et le place au début.

1. Écrire le pseudo-code.
2. Donner la trace pour $[5, 3, 8, 1, 9, 2]$.
3. Compter les comparaisons et les échanges.
4. Compter le nombre de comparaisons lorsque la liste contient $n$ valeurs, puis lorsque la liste est déjà triée.
5. Comparer avec le tri à bulles (exercice 2.31).

**★ Défi – hors programme T10, non exigible – *Récursivité : flocon de Koch***

Le *flocon de Koch* se construit ainsi : on part d'un segment de longueur $L$. À chaque étape, on remplace chaque segment par quatre segments, chacun de longueur $L/3$, en formant une « bosse » triangulaire.

1. Décrire en pseudo-code l'algorithme récursif qui affiche le nombre de segments et la longueur totale après $n$ étapes. *(Paramètres : longueur initiale, profondeur $n$.)*
2. Compléter le tableau :

| Étape $n$ | Nombre de segments | Longueur totale |
|---|---|---|
| 0 | 1 | $L$ |
| 1 | 4 | $4L/3$ |
| 2 | | |
| 3 | | |
| $n$ | | |

3. La longueur totale est-elle bornée quand $n \to +\infty$ ? Que se passe-t-il si on part d'un triangle équilatéral et qu'on applique la construction à chacun de ses côtés ?

4. Expliquer pourquoi un algorithme récursif sans condition d'arrêt serait problématique : que se passerait-il ?

### ★ Défi – hors programme T10, non exigible – *Problème des quatre reines*

Placer 4 reines sur un échiquier $4 \times 4$ sans qu'aucune ne menace une autre (pas deux sur la même ligne, la même colonne, ni la même diagonale).

1. Pourquoi est-il impossible de placer 2 reines sur un échiquier $2 \times 2$ ? Et 3 reines sur un $3 \times 3$ ?

2. Sur le $4 \times 4$, on place les reines une par colonne. La première reine est en colonne 1, ligne $a$. Pour chaque valeur de $a \in \{1, 2, 3, 4\}$, déterminer quelles lignes sont disponibles en colonne 2.

3. En continuant le raisonnement, trouver **à la main** toutes les solutions (il y en a exactement 2). Les dessiner sur des grilles $4 \times 4$.

4. Écrire en pseudo-code un algorithme qui essaie systématiquement toutes les positions possibles pour les 4 reines (une par colonne) et affiche les solutions valides. Combien d'essais au maximum ?

### ★ Défi – hors programme T10, non exigible – *Simulation d'un automate simple*

Un *distributeur de boissons* fonctionne comme suit : il accepte des pièces de 1 € ou 2 € et distribue une boisson quand le total atteint ou dépasse 3 €.

1. Écrire un algorithme qui simule ce distributeur : il lit des pièces une par une (valeur 1 ou 2) et affiche « Boisson distribuée ! » dès que la somme $\geq 3$. Il restitue l'excédent éventuel.

2. Donner la trace pour la séquence de pièces $1, 1, 2$. Puis pour $2, 2$.

3. Modifier l'algorithme pour refuser les pièces qui ne sont ni 1 € ni 2 € (afficher « Pièce invalide »).

4. Combien existe-t-il de séquences de pièces différentes aboutissant à exactement 3 € (sans excédent) ? Les lister systématiquement.

5. **Extension :** le distributeur a un *état interne* (montant cumulé). Représenter les états possibles et les transitions entre eux par un diagramme (cercles = états, flèches = pièces).